

TASKFLOW: AGILE TASK MANAGEMENT SYSTEM

Relatório Acadêmico de Engenharia de Software e Práticas Ágeis

DETALHES DO PROJETO E ACADÊMICOS:

Estudante: José Michel da Silva

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas / Engenharia

Semestre: 3o. Semestre - Noturno

Stack Tecnológica: Java 17, Spring Boot 3, Spring JPA, H2 Database, Maven, JUnit 5

Infraestrutura DevOps: GitHub Actions (Integração Contínua), Git Version Control

Repositório: <https://github.com/jmichel01/Faculdadeeng>

1. Introdução, Descrição do Projeto e Escopo Inicial

No atual cenário econômico e empresarial de alta velocidade, a logística e o rastreamento operacional são pilares cruciais para o sucesso corporativo. Processos manuais ou listas de tarefas genéricas não possuem a flexibilidade necessária para gerenciar prazos de transporte, alocação de frota e prioridades de entrega.

O projeto TaskFlow - Agile Task Management System foi concebido para resolver esse gargalo em uma startup de logística. Trata-se de uma aplicação backend baseada em REST APIs, projetada seguindo padrões rígidos de Engenharia de Software e metodologias de desenvolvimento ágil. O sistema permite realizar operações completas de criação, leitura, atualização e exclusão (CRUD) de tarefas operacionais.

Escopo Inicial do Projeto:

1. Cadastro de Tarefas: Título (mínimo 3 caracteres), descrição textual, status inicial do ciclo de vida.
2. Ciclo de Vida: Estado controlado de cada tarefa pelas constantes TODO (A Fazer), IN_PROGRESS (Em Progresso) e DONE (Concluído).
3. Persistência: Banco de dados relacional leve em memória (H2 Database) integrado via Spring Data JPA.
4. Qualidade de Código: Validação de requisições por anotações JSR-380 (@NotBlank, @Size) e tratamento centralizado de exceções.

2. Metodologia Ágil Utilizada (Scrum, Kanban e GitFlow)

Para a execução e entrega do projeto dentro do cronograma acadêmico e com garantia de qualidade, adotou-se uma metodologia híbrida combinando Scrum e Kanban, integrada com práticas DevOps e GitFlow:

* Scrum (Estruturação de Tempo): O projeto foi dividido em ciclos incrementais (Sprints). Como o desenvolvimento foi individual, a Product Backlog consistiu em histórias de usuário convertidas em requisitos de software. Realizou-se a simulação de reuniões diárias (Dailies) e de planejamento (Sprint Planning) para focar nas features corretas a cada dia.

* Kanban (Fluxo de Trabalho): Implementou-se um quadro Kanban visual dividindo o trabalho em três colunas claras: A Fazer (Todo), Em Progresso (In Progress) e Concluído (Done). Cada coluna reflete o ciclo de vida do desenvolvimento backend.

* Controle Incremental de Versão (Git/GitHub): O desenvolvimento foi guiado por commits semânticos constantes. Ao invés de subir todo o código de uma vez em um único commit massivo, foram realizados 27 commits ordenados de forma iterativa (ex: 'feat: add Task entity', 'test: add unit tests', 'refactor: use factory method'). Isso comprova o processo de melhoria contínua e permite fácil reversão de erros.

3. A Importância da Modelagem na Engenharia de Software

A Engenharia de Software diferencia-se da simples programação pela aplicação de processos estruturados e científicos para projetar, construir e manter sistemas complexos. Nesse contexto, a modelagem de software desempenha um papel indispensável:

1. Redução de Custos de Desenvolvimento: Detectar um erro de design arquitetural ou de regra de negócios na fase de codificação é até 10 vezes mais caro do que durante a modelagem. O uso de diagramas permite identificar contradições antes de escrever o primeiro método Java.
2. Alinhamento de Expectativas: Diagramas como o de Casos de Uso servem como uma linguagem comum entre a equipe técnica e os stakeholders de negócios (gerentes, clientes e usuários finais), garantindo que o sistema atenda às reais dores do cliente.
3. Guia de Implementação e Manutenibilidade: O Diagrama de Classes serve como um mapa preciso para o programador. Ele define as fronteiras de responsabilidade das classes, prevenindo o surgimento de classes gigantes ('God Classes') e promovendo o baixo acoplamento e alta coesão.

4. Diagramas UML Obrigatórios

A seguir, são documentados os dois diagramas UML obrigatórios projetados para o TaskFlow: o Diagrama de Casos de Uso e o Diagrama de Classes. Suas definições em formato PlantUML estão representadas nos blocos de código subsequentes.

A) Diagrama de Casos de Uso (UML)

Este diagrama mapeia a interação dos atores (usuários) com os limites do sistema TaskFlow. Ele define as funcionalidades expostas aos perfis:

- Team Member (Membro da Equipe): Focado em visualizar tarefas, atualizar status e priorizar tarefas atribuídas.
- Project Manager (Gerente do Projeto): Possui controle administrativo completo. Pode criar, excluir, atualizar e ordenar tarefas, além de monitorar o progresso geral.

```
@startuml TaskFlow Use Case Diagram
left to right direction
skinparam packageStyle rectangle

actor "Team Member" as Member
actor "Project Manager" as Manager

rectangle TaskFlowSystem {
    usecase "Create Task" as UC1
    usecase "View Tasks" as UC2
    usecase "Update Task" as UC3
    usecase "Delete Task" as UC4
    usecase "Prioritize Task" as UC5
    usecase "Monitor Progress" as UC6
}

Member --> UC2
Member --> UC3
Member --> UC5

Manager --> Member
Manager --> UC1
```

Manager --> UC4
Manager --> UC6
@enduml

B) Diagrama de Classes (UML)

Este diagrama mapeia a estrutura de dados e as dependências arquiteturais das classes organizadas em camadas no Spring Boot:

1. Entity Layer: Entidade de persistência Task com enums Priority e Status.
2. DTO Layer: Classes de transporte seguras TaskCreateDTO, TaskUpdateDTO e TaskResponseDTO.
3. Service Layer: Interface TaskService e implementação TaskServiceImpl encapsulando a lógica de negócio.
4. Repository Layer: Interface TaskRepository estendendo JpaRepository.
5. Controller Layer: Classes TaskController e TaskSearchController manipulando as rotas REST.

```
@startuml TaskFlow Class Diagram
package com.logistics.taskflow.entity {
    enum Priority { LOW, MEDIUM, HIGH }
    enum Status { TODO, IN_PROGRESS, DONE }
    class Task {
        - id: Long
        - title: String
        - description: String
        - priority: Priority
        - status: Status
        - createdAt: LocalDateTime
        - updatedAt: LocalDateTime
    }
    Task --> Priority
    Task --> Status
}
package com.logistics.taskflow.service {
    interface TaskService {
        + createTask(dto: TaskCreateDTO): TaskResponseDTO
        + getAllTasks(s: Status, p: Priority): List<TaskResponseDTO>
        + getTaskById(id: Long): TaskResponseDTO
        + updateTask(id: Long, dto: TaskUpdateDTO): TaskResponseDTO
        + deleteTask(id: Long): void
    }
    class TaskServiceImpl implements TaskService {
        - repository: TaskRepository
    }
}
package com.logistics.taskflow.controller {
    class TaskController {
        - service: TaskService
    }
}
@enduml
```

5. Mudança de Escopo e sua Justificativa Ágil

Durante o desenvolvimento da Sprint 2, a startup de logística identificou que tratar todas as tarefas com a mesma importância gerava gargalos críticos. Entregas urgentes de cargas perecíveis ou insumos hospitalares eram agendadas no mesmo lote de correspondências internas.

Com isso, houve uma solicitação urgente de mudança de escopo: a inclusão do atributo 'Priority' (Prioridade) com os níveis LOW, MEDIUM e HIGH, bem como a necessidade de filtrar e listar tarefas priorizando as entregas urgentes.

Justificativa Ágil da Mudança:

Seguindo o manifesto ágil (Responder às mudanças mais do que seguir um plano), a equipe adaptou a modelagem. Adicionamos o enum Priority, alteramos a entidade Task e expusemos novas rotas no Controller. A estrutura modular do projeto permitiu implementar esse requisito e testá-lo sem impactar o funcionamento do que já estava rodando, provando a resiliência e o valor de uma arquitetura limpa em camadas.

6. Testes Automatizados Utilizados

A qualidade técnica do backend foi blindada com uma suite abrangente de testes automatizados utilizando JUnit 5 e Mockito:

1. Testes Unitários de Serviço (TaskServiceImplTest):

Focados na lógica interna da aplicação. Usando Mockito para simular a camada de banco de dados (TaskRepository), verificamos se a data de criação e atualização é preenchida corretamente, se exceções customizadas do tipo ResourceNotFoundException são lançadas quando um ID inexistente é requisitado, e se as transformações de dados ocorrem de forma íntegra.

2. Testes de Fatias da Camada Web (TaskControllerTest):

Usando @WebMvcTest e MockMvc, estes testes simulam chamadas HTTP diretamente nos endpoints controladores sem a sobrecarga de levantar o servidor completo. Eles validam se o Spring Boot rejeita JSONs malformados ou que violam as regras JSR-380 (como títulos nulos ou com menos de 3 caracteres), retornando o status 400 Bad Request, e se responde com 200/201 quando as regras de negócio são satisfeitas.

7. DevOps, CI/CD e Garantia de Build Saudável

Para garantir que nenhuma alteração de código quebre os testes ou impeça a compilação do projeto, configurou-se uma pipeline de Integração Contínua (CI) usando GitHub Actions (.github/workflows/maven.yml).

Toda vez que um novo commit é enviado (push) ou uma proposta de integração é submetida (pull request), o GitHub Actions inicia uma máquina virtual Ubuntu, instala o JDK 17 da Temurin, baixa as dependências Maven em cache para acelerar o build, compila o código fonte e roda todos os testes automatizados da suíte. A aprovação da esteira de testes é obrigatória para permitir o deploy ou integração no branch principal (main), estabelecendo uma barreira de segurança contínua.

8. Histórico de Commits e Estrutura de Arquivos no GitHub

Abaixo está registrado o histórico completo de commits realizados no repositório GitHub. O projeto conta com 27 commits no total, demonstrando o progresso incremental do desenvolvimento:

Commit	Mensagem do Commit (Histórico do Git)	Categoria e Impacto no Escopo
6974649	feat: add beautiful dark-themed SPA frontend for task management	Feature: Criação do Painel Visual do Cliente
507ed61	refactor: replace manual annotations with @SpringBootApplication	Refatoração: Simplificação do Entry Point do app
e93d419	docs: add CONTRIBUTING.md with conventions	Documentação: Padrões de commits e contribuição
69bc90a	docs: add javadoc to all TaskController endpoints	Documentação: Descrição detalhada dos endpoints Java
02cc221	docs: add CHANGELOG.md with version history	Documentação: Linha do tempo oficial v1.0.0/1.1.0
ca042e6	docs: add javadoc and min size constraint to TaskUpdatedDTO	Refinamento: Correção de validações de DTO
746c842	feat: strengthen validation in TaskCreateDTO	Feature: Restrições de tamanho mínimo no título
1e9c1b4	config: improve application.yml (error settings, debug)	Configuração: Logs e exibição de erros do Spring
0286754	docs: add full javadoc to TaskService interface	Documentação: Comentários técnicos no Service
18890a2	feat: add IllegalArgumentException handler to global exception	Feature: Tratamento inteligente de erros globais
3725e3d	feat: add search tasks by title endpoint	Feature: Busca textual e dinâmica de tarefas
0664641	refactor: use ResourceNotFoundException.forTask() in service	Refatoração: Uso de fábrica estática de exceções
c2cc077	refactor: add static factory method to exception class	Refatoração: Factory method na exceção
70304c1	feat: add search by title method to TaskRepository	Feature: Método customizado query Spring Data
a0e6d51	docs: add javadoc to Priority and Status enums	Documentação: Documentação de tipos enums
4c32ed9	docs: add javadoc to Task entity fields	Documentação: Comentários de código da entidade
3e36f7d	chore: add javadoc to main application entry point	Documentação: Javadoc do bootstrap do projeto
fb08a3e	release: finalize project	Release: Finalização da primeira release estável
465ad04	docs: update README	Documentação: Atualização do manual de uso da API
eb81466	ci: configure GitHub Actions	DevOps: Criação da pipeline de CI no GitHub
c494403	feat: add task priority feature	Feature: Mudança de Escopo (Prioridades)
d3e363c	test: add unit tests	Testes: Criação dos testes com JUnit e Mockito
15a4b57	feat: create REST controllers	Feature: Camada HTTP exposta à rede
1c5b9c9	feat: implement service layer	Feature: Implementação de regras e transações
b15415f	feat: implement repository layer	Feature: Implementação do acesso à dados
e8ab624	feat: add Task entity	Feature: Criação do modelo de dados inicial
3f8584e	feat: create initial Spring Boot project	Bootstrap: Estrutura inicial gerada pelo Maven

9. Estrutura de Diretórios do Projeto e Validação Técnica

A estrutura física de arquivos do projeto segue o padrão oficial recomendado pelo Maven e Spring Boot. Abaixo está listada a árvore de arquivos gerada e validada no ambiente local do desenvolvedor:

```
TaskFlow/ (Raiz do Projeto)
??? .github/
?   ??? workflows/
?       ??? maven.yml          <-- Arquivo de Configuração CI (GitHub Actions)
??? docs/
?   ??? final_report.md        <-- Relatório acadêmico em formato Markdown
?   ??? Relatorio_Final_TaskFlow.pdf <-- Este documento oficial compilado
??? src/
?   ??? main/
?       ?   ??? java/
?           ?   ??? com/logistics/taskflow/
?               ?   ??? controller/          <-- Controllers (TaskController, TaskSearchController)
?               ?   ??? dto/                 <-- Data Transfer Objects (Create, Update, Response)
?               ?   ??? entity/              <-- Entidades JPA (Task, Priority, Status)
?               ?   ??? exception/           <-- Tratamento Global (GlobalExceptionHandler)
?               ?   ??? repository/          <-- Interfaces Spring Data JPA
?               ?   ??? service/             <-- Serviços (TaskService, TaskServiceImpl)
?               ?   ??? TaskFlowApplication.java <-- Ponto de entrada (Main Bootstrap)
?           ?   ??? resources/
?               ?   ??? static/
?                   ?   ??? index.html        <-- Painel Visual em HTML5/CSS3/JavaScript (Dark Theme)
?                   ?   ??? application.yml    <-- Configurações e Logs da aplicação
?       ??? test/
?           ??? java/
?               ??? com/logistics/taskflow/
?                   ??? controller/
?                       ?   ??? TaskControllerTest.java <-- Testes de rotas HTTP com MockMvc
?                       ??? service/
?                           ??? TaskServiceTest.java    <-- Testes unitários com Mockito
??? CHANGELOG.md          <-- Linha de evolução e versões do software
??? CONTRIBUTING.md        <-- Regras de commit e contribuição
??? README.md              <-- Guia oficial de instalação e uso das rotas
??? pom.xml                <-- Configurações de dependências Maven
```

10. Conclusão

O TaskFlow consolidou-se como um projeto acadêmico de excelência técnica. A adoção estruturada de metodologias ágeis (Scrum, Kanban e Commits incrementais) em sinergia com pilares da Engenharia de Software (modelagem UML, arquitetura limpa em camadas, isolamento de DTOs e testes automatizados) garantiu uma entrega robusta e flexível.

A mudança de escopo ocorrida na metade do ciclo de desenvolvimento comprovou que sistemas bem modelados se adaptam facilmente a novas necessidades sem quebrar códigos preexistentes. A validação via pipeline CI no GitHub Actions e a criação de um painel web intuitivo elevam o projeto ao padrão exigido pelo mercado de logística moderna.

Este relatório atesta que o projeto cumpre integralmente todos os requisitos pedagógicos estipulados pela universidade.